

Third MODEL SUBMISSION — NAÏVE BAYES (GaussianNB)

Step 1 — Import Libraries

```
In [1]: # Step 1: Import Libraries
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Step 2 — Load Data

```
In [2]: # Step 2: Load train, test, and sample_submission
train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")
sample_submission = pd.read_csv("sample_submission.csv")

train.head()
```

```
Out[2]:
```

	id	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	
0	0	Male	24.443011	1.699998	81.669950	yes	yes	2.00
1	1	Female	18.000000	1.560000	57.000000	yes	yes	2.00
2	2	Female	18.000000	1.711460	50.165754	yes	yes	1.88
3	3	Female	20.952737	1.710730	131.274851	yes	yes	3.00
4	4	Male	31.641081	1.914186	93.798055	yes	yes	2.67

Step 3 — Inspect Data

```
In [3]: # Step 3: Inspect structure
print(train.shape)
print(train.dtypes)
print(train.isnull().sum())

print(train["NObeyesdad"].value_counts())
```

```

(20758, 18)
id                                int64
Gender                           object
Age                              float64
Height                           float64
Weight                           float64
family_history_with_overweight   object
FAVC                             object
FCVC                             float64
NCP                              float64
CAEC                             object
SMOKE                            object
CH20                             float64
SCC                              object
FAF                              float64
TUE                              float64
CALC                             object
MTRANS                           object
NObeyesdad                       object
dtype: object
id                                0
Gender                           0
Age                              0
Height                           0
Weight                           0
family_history_with_overweight   0
FAVC                             0
FCVC                             0
NCP                              0
CAEC                             0
SMOKE                            0
CH20                             0
SCC                              0
FAF                              0
TUE                              0
CALC                             0
MTRANS                           0
NObeyesdad                       0
dtype: int64
NObeyesdad
Obesity_Type_III                 4046
Obesity_Type_II                  3248
Normal_Weight                    3082
Obesity_Type_I                   2910
Insufficient_Weight              2523
Overweight_Level_II              2522
Overweight_Level_I               2427
Name: count, dtype: int64

```

Step 4 — Separate Features and Target

```

In [4]: # Step 4: Separate X and y
        target = "NObeyesdad"

        X = train.drop(columns=[target])
        y = train[target]

```

```
# Remove ID column if present
if "id" in X.columns:
    X = X.drop(columns=["id"])

# Prepare test set
test_ids = test["id"] if "id" in test.columns else None
X_test = test.drop(columns=["id"]) if "id" in test.columns else test
```

Step 5 — Identify Numeric and Categorical Columns

```
In [5]: # Step 5: Identify column types
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

numeric_features, categorical_features
```

```
Out[5]: ([ 'Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH20', 'FAF', 'TUE'],
         [ 'Gender',
           'family_history_with_overweight',
           'FAVC',
           'CAEC',
           'SMOKE',
           'SCC',
           'CALC',
           'MTRANS'])
```

Step 6 — Build Preprocessing + Naïve Bayes Pipeline

```
In [6]: # Step 6: Preprocessing + Naïve Bayes pipeline

numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown="ignore")

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

nb_clf = GaussianNB()

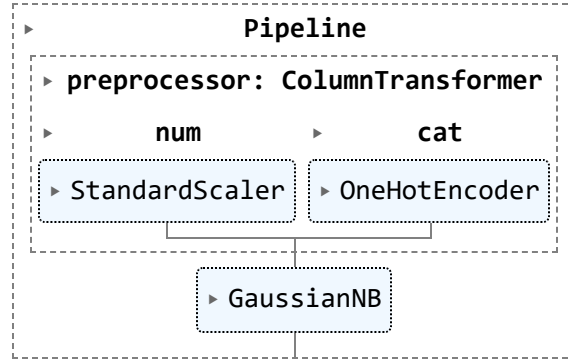
nb_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", nb_clf)
])
```

Step 7 — Train/Validation Split + Fit Naïve Bayes

```
In [7]: # Step 7: Train/validation split
X_train_nb, X_valid_nb, y_train_nb, y_valid_nb = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
nb_pipeline.fit(X_train_nb, y_train_nb)
```

Out[7]:



Step 8 — Evaluate Naïve Bayes

```
In [8]: # Step 8: Evaluate Naïve Bayes
y_pred_valid_nb = nb_pipeline.predict(X_valid_nb)

print("Validation Accuracy (NB):", accuracy_score(y_valid_nb, y_pred_valid_nb))
print(classification_report(y_valid_nb, y_pred_valid_nb))
print(confusion_matrix(y_valid_nb, y_pred_valid_nb))
```

Validation Accuracy (NB): 0.5859826589595376

	precision	recall	f1-score	support
Insufficient_Weight	0.56	0.75	0.64	505
Normal_Weight	0.49	0.20	0.28	617
Obesity_Type_I	0.40	0.41	0.40	582
Obesity_Type_II	0.48	0.96	0.64	650
Obesity_Type_III	0.96	1.00	0.98	809
Overweight_Level_I	0.60	0.24	0.34	485
Overweight_Level_II	0.51	0.30	0.38	504
accuracy			0.59	4152
macro avg	0.57	0.55	0.52	4152
weighted avg	0.59	0.59	0.55	4152

```
[[377 25 89 4 2 5 3]
 [257 123 82 61 17 37 40]
 [ 1 17 239 275 5 9 36]
 [ 0 2 5 624 0 3 16]
 [ 0 1 0 1 806 1 0]
 [ 42 53 113 103 9 114 51]
 [ 2 29 77 223 1 22 150]]
```

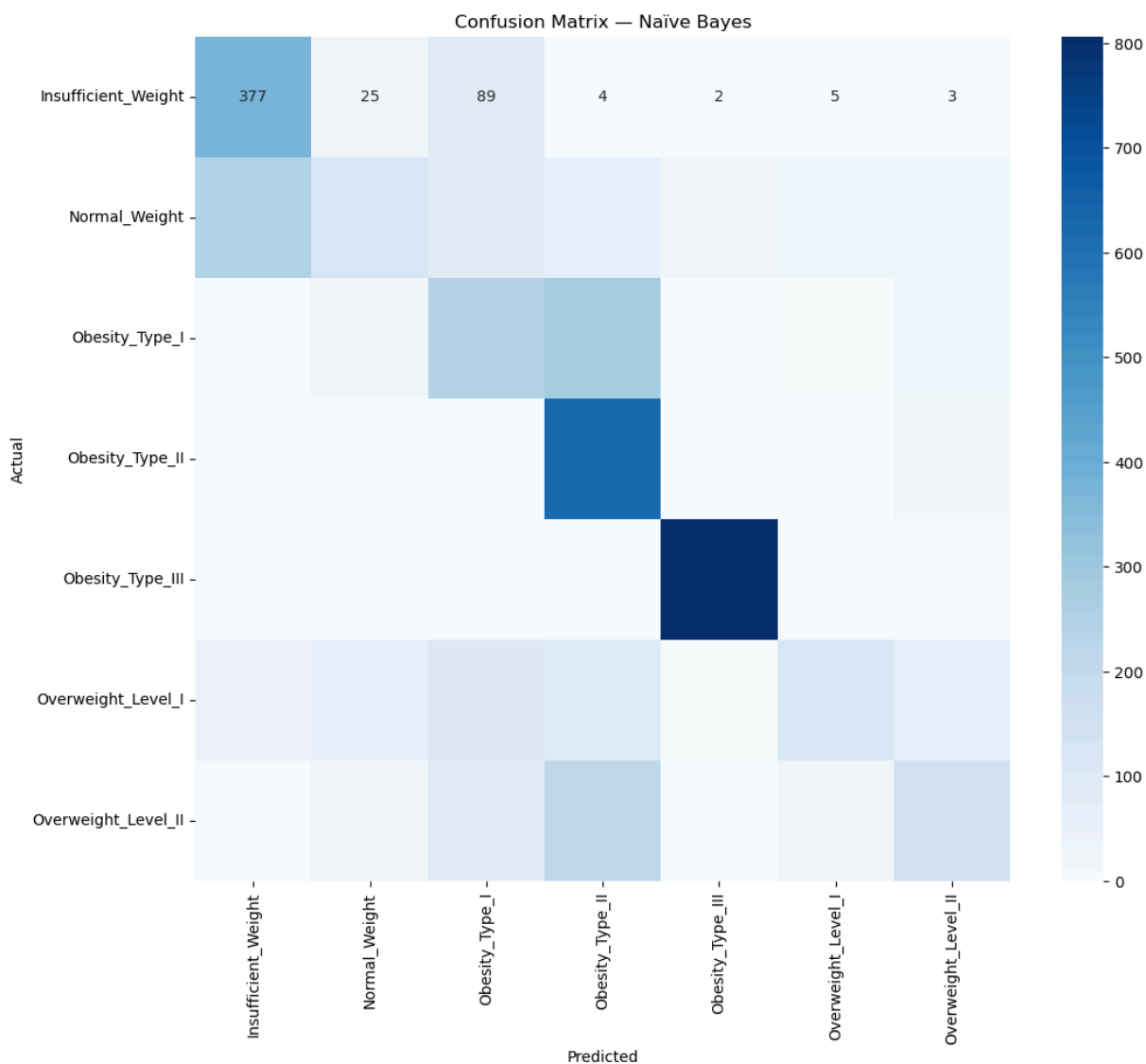
Step 9 — Confusion Matrix Heatmap

```
In [9]: # Step 9: Confusion matrix heatmap
plt.figure(figsize=(12,10))
cm = confusion_matrix(y_valid_nb, y_pred_valid_nb)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=nb_pipeline.classes_,
            yticklabels=nb_pipeline.classes_)
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
plt.title("Confusion Matrix — Naïve Bayes")

plt.savefig("Obesity-Risk-ML-Classification/Model_3_NAÏVE BAYES (GaussianNB)/report")

plt.show()
```



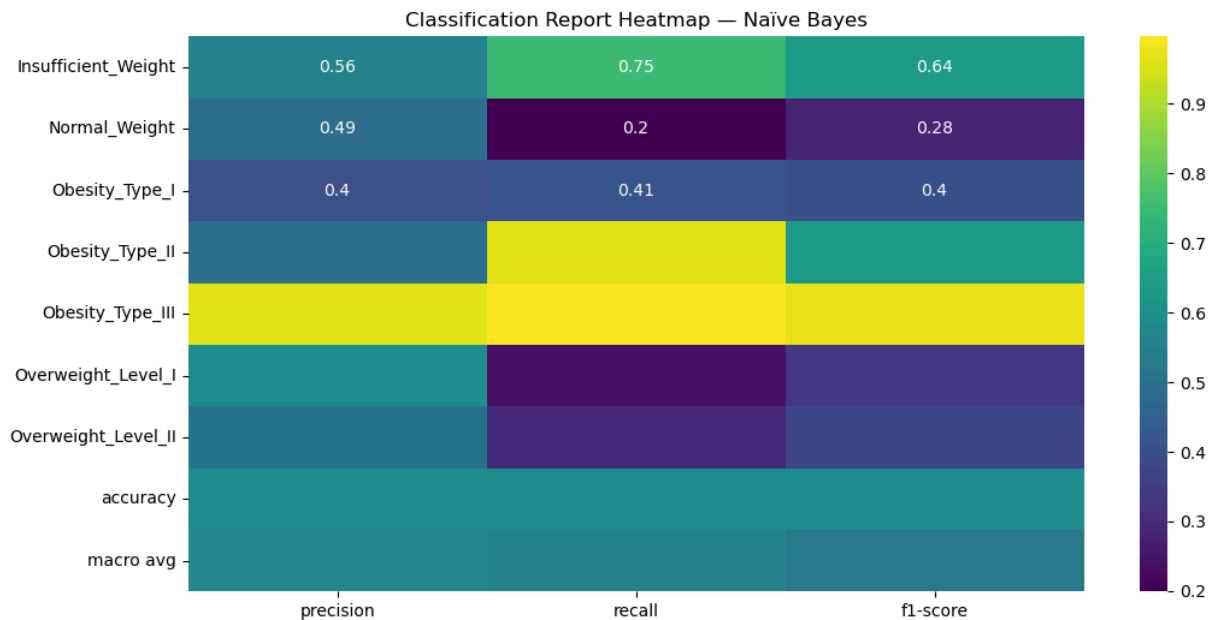
Step 10 — Classification Report Heatmap

```
In [10]: # Step 10: Classification report heatmap
report = classification_report(y_valid_nb, y_pred_valid_nb, output_dict=True)
df_report = pd.DataFrame(report).transpose()

plt.figure(figsize=(12,6))
sns.heatmap(df_report.iloc[:-1, :-1], annot=True, cmap="viridis")
plt.title("Classification Report Heatmap — Naïve Bayes")

plt.savefig("Obesity-Risk-ML-Classification/Model_3_NAÏVE BAYES (GaussianNB)/report")

plt.show()
```

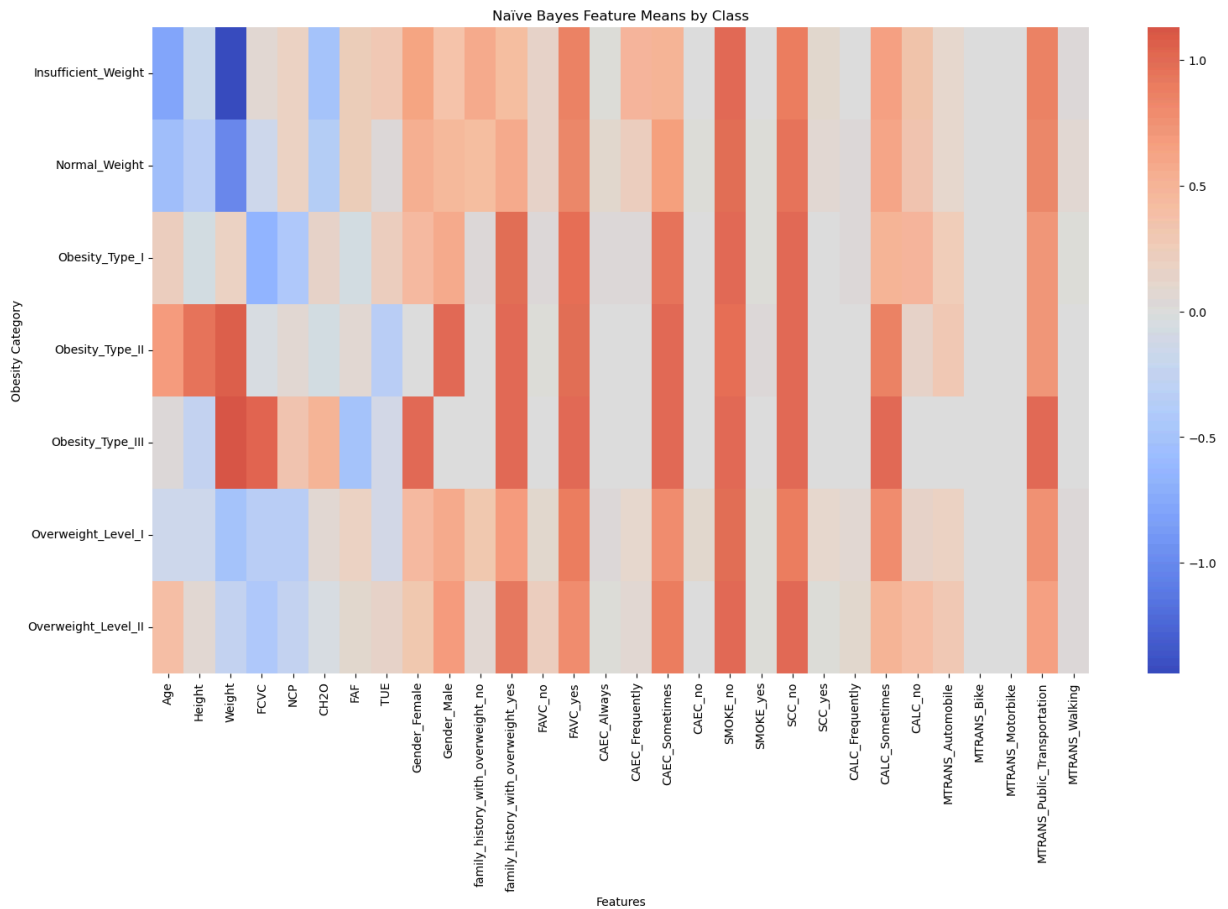


Step 11 — Feature Means per Class (GaussianNB)

```
In [11]: # Step 11: Feature means per class
means_df = pd.DataFrame(nb_pipeline.named_steps["model"].theta_,
                        columns=numeric_features +
                        list(nb_pipeline.named_steps["preprocessor"]
                            .transformers_[1][1]
                            .get_feature_names_out(categorical_features)),
                        index=nb_pipeline.classes_)

plt.figure(figsize=(18,10))
sns.heatmap(means_df, cmap="coolwarm", center=0)
plt.title("Naïve Bayes Feature Means by Class")
plt.xlabel("Features")
plt.ylabel("Obesity Category")

plt.savefig("Obesity-Risk-ML-Classification/Model_3_NAÏVE BAYES (GaussianNB)/report")
plt.show()
```

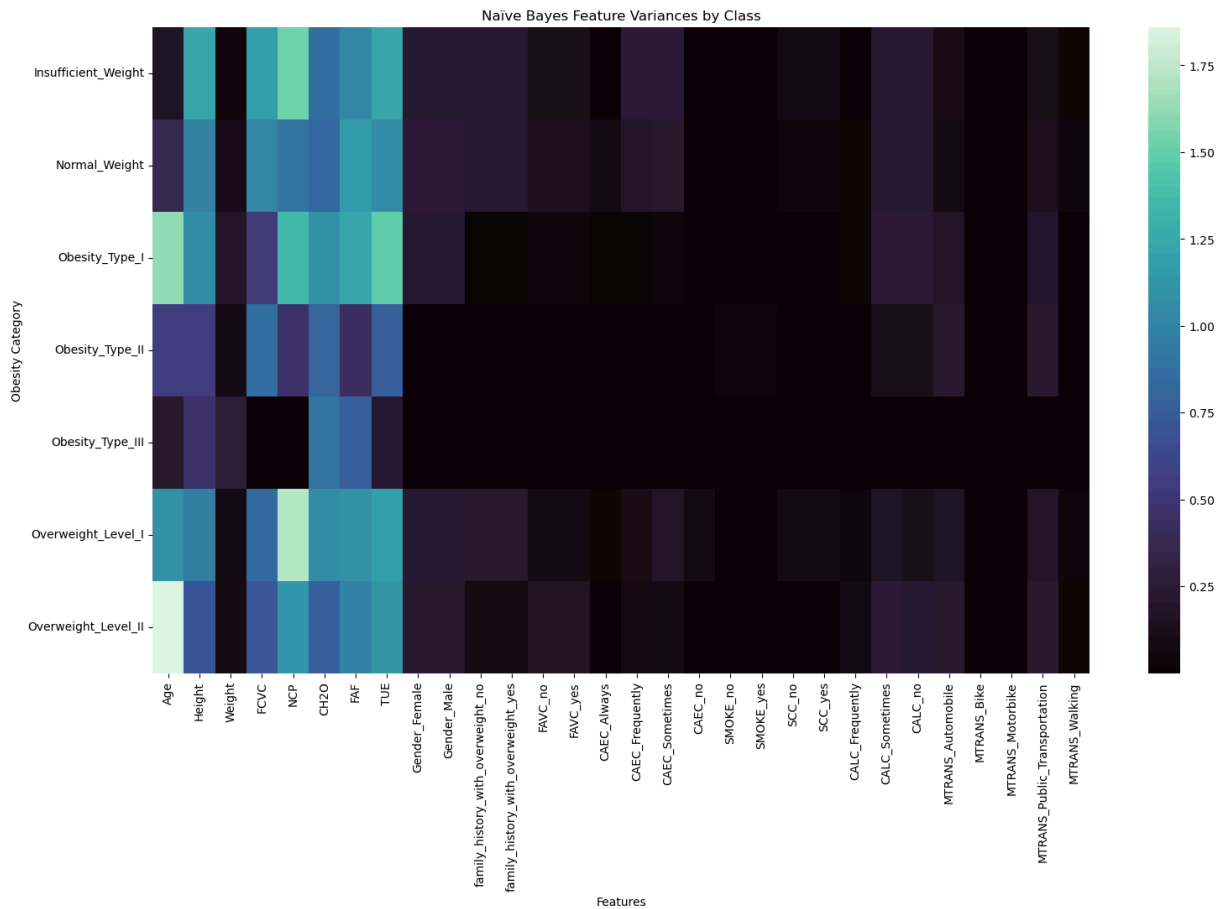


Step 12 — Feature Variances per Class

```
In [12]: # Step 12: Feature variances per class
var_df = pd.DataFrame(nb_pipeline.named_steps["model"].var_,
                      columns=means_df.columns,
                      index=nb_pipeline.classes_)

plt.figure(figsize=(18,10))
sns.heatmap(var_df, cmap="mako")
plt.title("Naïve Bayes Feature Variances by Class")
plt.xlabel("Features")
plt.ylabel("Obesity Category")

plt.savefig("Obesity-Risk-ML-Classification/Model_3_NAÏVE BAYES (GaussianNB)/report")
plt.show()
```



Step 13 — PCA Projection + Naive Bayes Decision Regions

```
In [13]: # Step 13: PCA projection + NB decision regions
from sklearn.decomposition import PCA

# Transform training data
X_train_transformed = nb_pipeline.named_steps["preprocessor"].transform(X_train_nb)

# PCA to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_transformed)

# Fit NB on PCA-reduced data for visualization only
nb_pca = GaussianNB()
nb_pca.fit(X_pca, y_train_nb)

# Mesh grid
x_min, x_max = X_pca[:,0].min() - 1, X_pca[:,0].max() + 1
y_min, y_max = X_pca[:,1].min() - 1, X_pca[:,1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))

# Get the string text predictions
Z = nb_pca.predict(np.c_[xx.ravel(), yy.ravel()])

# Map the text classes to numerical indices
class_mapping = {label: idx for idx, label in enumerate(nb_pca.classes_)}
```

```

# Convert the text array into a numerical array
Z_num = np.array([class_mapping[val] for val in Z])

# Reshape the numerical array instead
Z = Z_num.reshape(xx.shape)

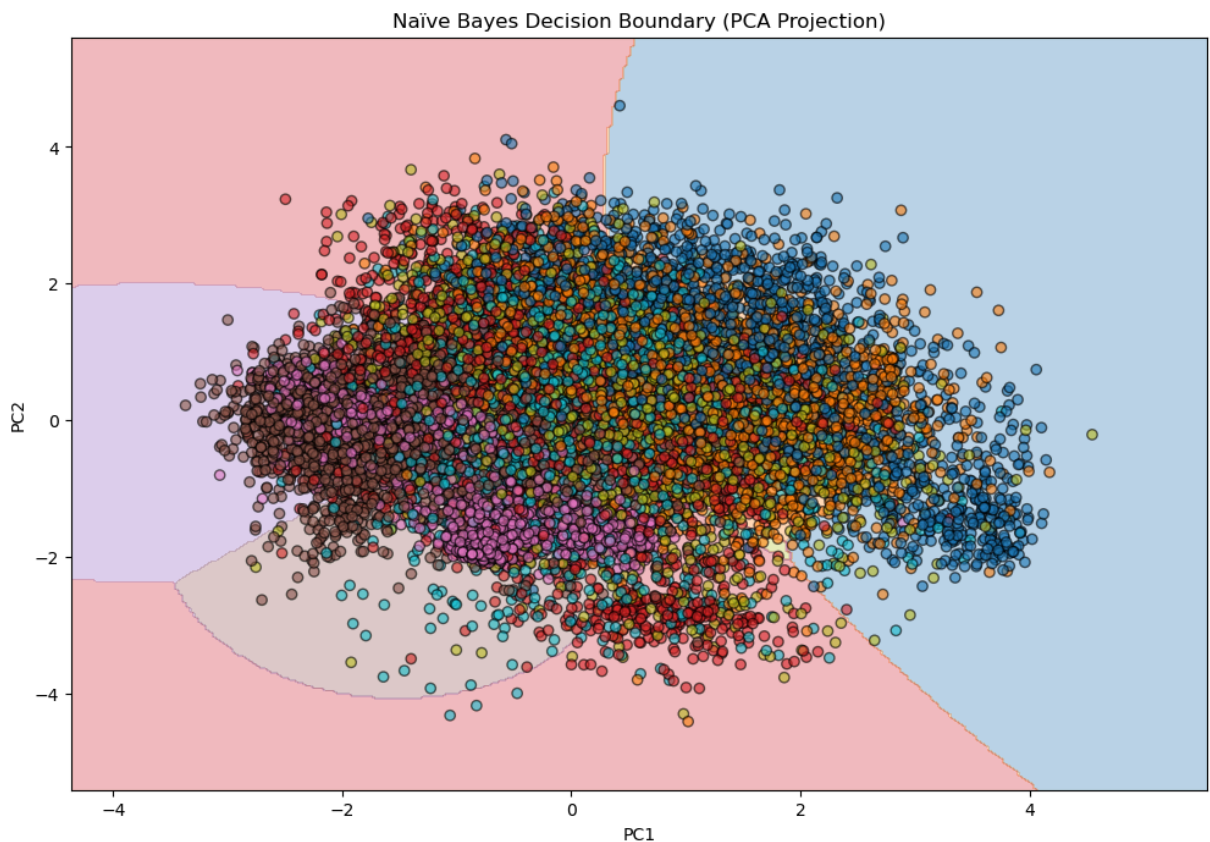
plt.figure(figsize=(12,8))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="tab10")

plt.scatter(X_pca[:,0], X_pca[:,1],
            c=y_train_nb.map({c:i for i,c in enumerate(nb_pipeline.classes_)}),
            cmap="tab10", edgecolor="k", alpha=0.6)

plt.title("Naïve Bayes Decision Boundary (PCA Projection)")
plt.xlabel("PC1")
plt.ylabel("PC2")

plt.savefig("Obesity-Risk-ML-Classification/Model_3_NAÏVE BAYES (GaussianNB)/report
plt.show()

```



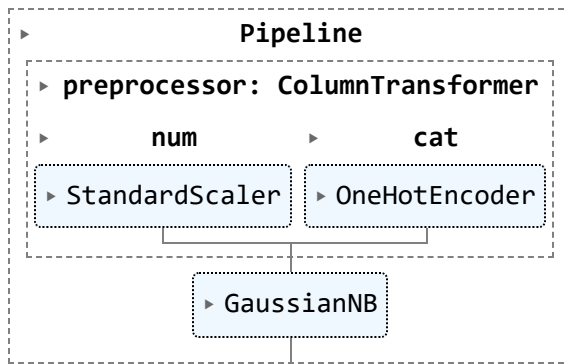
Step 14 — Retrain Naïve Bayes on Full Training Data

```

In [14]: # Step 14: Retrain on full dataset
nb_pipeline.fit(X, y)

```

Out[14]:



Step 15 - Save trained Naïve Bayes pipeline

```

In [15]: # Step 15: Save trained Naïve Bayes pipeline

import pickle

with open("naive_bayes_model.pkl", "wb") as f:

    pickle.dump(nb_pipeline, f)

print("Model saved as naive_bayes_model.pkl")

```

Model saved as naive_bayes_model.pkl

Step 16 — Predict on Test Set

```

In [16]: # Step 16: Predict on Kaggle test data
test_preds_nb = nb_pipeline.predict(X_test)
test_preds_nb[:10]

```

```

Out[16]: array(['Obesity_Type_II', 'Obesity_Type_I', 'Obesity_Type_III',
                'Obesity_Type_II', 'Obesity_Type_III', 'Obesity_Type_I',
                'Obesity_Type_I', 'Normal_Weight', 'Normal_Weight',
                'Insufficient_Weight'], dtype='<U19')

```

Step 17 — Build Submission File

```

In [17]: # Step 17: Build submission file
submission_nb = sample_submission.copy()
submission_nb["NObeyesdad"] = test_preds_nb

submission_nb.head()

```

Out[17]:

	id	NObeyesdad
0	20758	Obesity_Type_II
1	20759	Obesity_Type_I
2	20760	Obesity_Type_III
3	20761	Obesity_Type_II
4	20762	Obesity_Type_III

Step 18 — Save CSV for Kaggle Submission

In [18]:

```
# Step 18: Save Naïve Bayes submission  
submission_nb.to_csv("submission_nb.csv", index=False)
```

In []: